

1.Introduction to pointers. Call by Value and Call by reference.

1. Introduction to Pointers

✓ What is a Pointer?

A **pointer** is a variable that **stores the address of another variable** instead of storing a value directly.

📌 Definition

A pointer is a variable that stores the memory address of another variable.

✓ Why Pointers are Needed?

- To access memory directly
 - To pass data efficiently to functions
 - To work with arrays, strings, and dynamic memory
 - Essential for **Data Structures** (linked list, stack, queue, tree)
-

✓ Declaration of Pointer

```
int *p;
```

- `int` → data type of variable whose address is stored
 - `*` → pointer symbol
 - `p` → pointer variable
-

✓ Example of Pointer

```
int a = 10;
int *p;
p = &a;
```

Expression	Meaning
a	Value of a (10)
&a	Address of a
p	Address stored in pointer
*p	Value at that address (10)

✓ Advantages of Pointers

- Efficient memory usage
 - Faster execution
 - Supports dynamic memory allocation
 - Required for complex data structures
-

2. Call by Value

✓ Definition

In **Call by Value**, **copy of actual value** is passed to the function.

✂ **Original value does NOT change.**

✓ Example

```
#include <stdio.h>

void change(int x) {
    x = 20;
}

int main() {
    int a = 10;
    change(a);
    printf("%d", a);
    return 0;
}
```

✓ Output

✓ Explanation

- Value of `a` is copied into `x`
 - Changes happen only inside function
 - Original variable remains unchanged
-

✓ Diagram

```
a = 10  → copied → x = 10
x = 20  (only inside function)
```

✓ Advantages

- Simple and safe
 - No risk of changing original data
-

✗ Disadvantages

- Cannot modify original variable
 - More memory usage
-

3. Call by Reference

✓ Definition

In **Call by Reference**, **address of variable** is passed to the function.

📌 **Original value CAN be changed.**

✓ Example using Pointer

```
#include <stdio.h>
```

```
void change(int *x) {
    *x = 20;
}

int main() {
    int a = 10;
    change(&a);
    printf("%d", a);
    return 0;
}
```

✓ Output

20

✓ Explanation

- Address of `a` is passed
 - Function accesses original memory
 - Value changes permanently
-

✓ Diagram

`a = 10`
`&a → pointer x`
`*x = 20 → a becomes 20`

✓ Advantages

- Changes original value
 - Efficient memory usage
 - Used in swapping, arrays, structures
-

✗ Disadvantages

- Less safe
 - Risk of unintended changes
-

4. Difference Between Call by Value & Call by Reference

Call by Value	Call by Reference
Copy of value passed	Address passed
Original value unchanged	Original value changed
More memory used	Less memory
Safer	Risky
No pointers used	Uses pointers

5. Important Exam Notes

- ✓ Pointer stores address
- ✓ * used for value at address
- ✓ & used to get address
- ✓ Call by Reference uses pointers
- ✓ Used heavily in Data Structures